

Департамент образования и науки города Москвы
Государственное автономное образовательное учреждение
высшего образования города Москвы
«Московский городской педагогический университет»
Институт цифрового образования
Департамент информатики управления и технологий

Мошенина Елена Дмитриевна БД-241м

Инструменты хранения и анализа больших данных

Практическая работа 3.1. "Машинное обучение на больших данных с использованием Apache Spark MLlib".

Направление подготовки/специальность
38.04.05 - Бизнес-информатика
Бизнес-аналитика и большие данные
(очная форма обучения)

Руководитель дисциплины:
Босенко Т.М., доцент департамента
информатики, управления и технологий,
доктор экономических наук

Москва
2025

Содержание

Введение 3

Основная часть 3

Заключение..... 43

Введение

Цель и задачи работы:

- Познакомиться с понятием «большие данные» и способами их обработки;
- Познакомиться с инструментом Apache Spark и возможностями, которые он предоставляет для обработки больших данных.
- Получить навыки выполнения разведочного анализа данных использованием ruyspark.

Основная часть

Порядок выполнения работы:

1. Запустите конфигурацию ds_mgpu_Hadoop3+spark_3_4 [скачать](#) или запустить [файл](#) в colab.
2. Клонировать текущий репозиторий на компьютер в случае работы локально в Docker:

\$ git clone <https://github.com/BosenkoTM/PySpark.git>

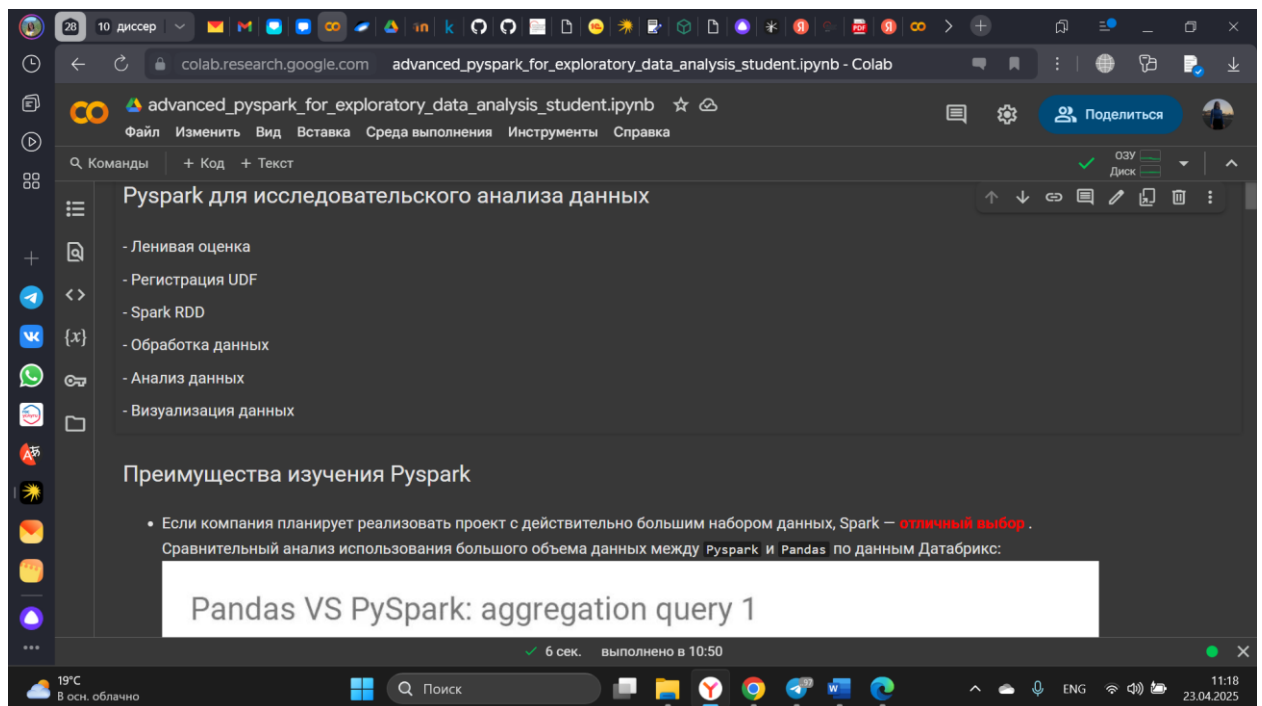
3. Скачайте датасет, расположенный по адресу https://drive.google.com/file/d/1yiAp1fFDy3wSqUR0X_btCZPtuczblwCe/view?usp=drive_link или [data](#). Распакуйте его и поместите

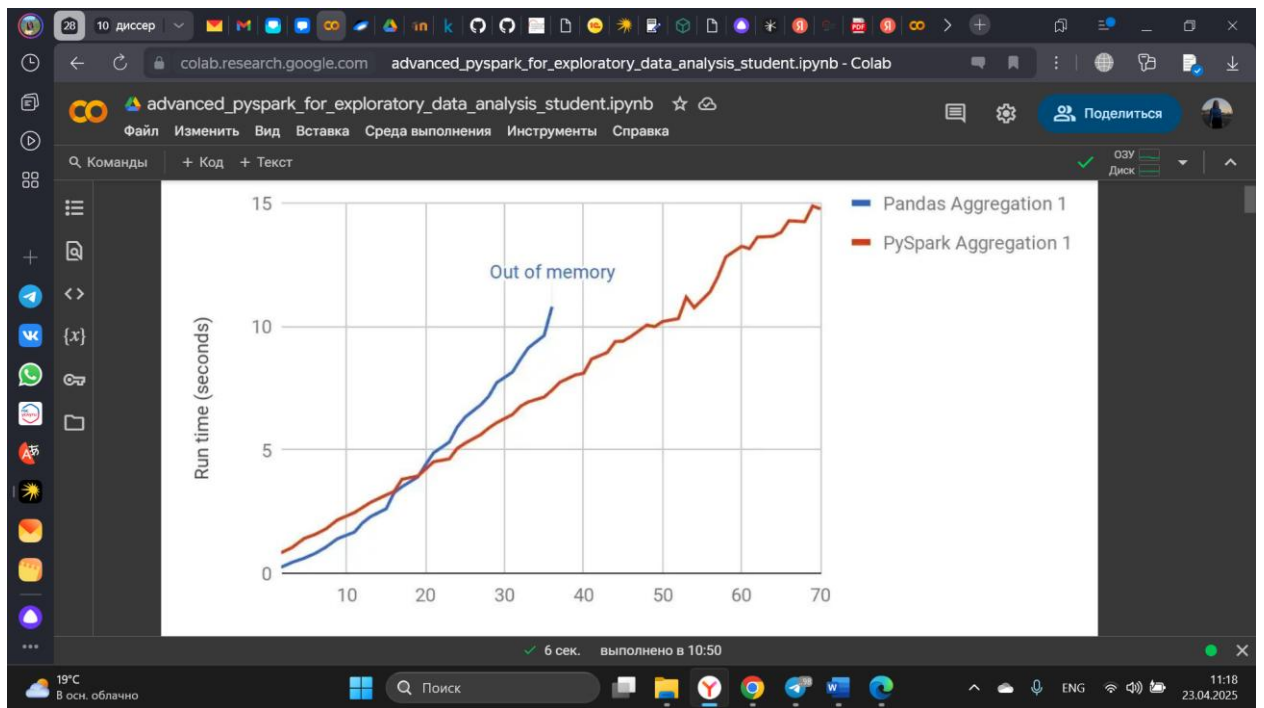
файл endomondoHR.json в директорию data проекта (если директория отсутствует, создайте ее).

4. Не Colab: запустите докер-контейнер со средой разработки и инструментом Apache Spark и дождитесь завершения его работы.

\$ sudo docker compose up

- Откройте браузер и перейдите по адресу: <http://localhost:10000/lab>
5. Откройте в браузере файл ноутбука с примером разведочного анализа [advanced_pyspark_for_exploratory_data_analysis_student.ipynb](#). Изучите этот файл и запустите его.





advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

Если вы планируете устроиться на работу в компанию с действительно большой экосистемой данных, знание Spark будет хорошим плюсом и отличает вас от других соискателей.

Ознакомиться с экосистемой Spark: <https://www.kaggle.com/tientd95/pyspark-for-data-science>

или скачать файл `pyspark-for-data-science.ipynb`

1. Инициализация платформы Spark и загрузка данных в фрейм данных Spark

[Go back to table of contents](#)

Источник данных 1: https://drive.google.com/file/d/1yiAp1fDy3wSqUR0X_btCZPtuczbLwCe/view?usp=drive_link

Источник данных 2: <https://disk.yandex.ru/d/HKtMWSuqglgaXg>

Этот набор данных предназначен для расчета частоты пульса у людей, а также других связанных с этим характеристик: пола, погодных условий, вида спорта, GPS и т.д.

```
[1] # Импорт модулей, не связанных с PySpark.
import os
import sys
```

6 сек. выполнено в 10:50

19°C В осн. облачно

Поиск

11:19 23.04.2025

```
[1] # Импортируем модули, не связанные с PySpark.
import os
import sys
import pandas as pd
from pandas import DataFrame
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import matplotlib
from mpl_toolkits.mplot3d import Axes3D
import math
from IPython.core.interactiveshell import InteractiveShell
from datetime import *
import statistics as stats
# Это помогает автоматически распечатывать элементы без явного использования «печати».
InteractiveShell.ast_node_interactivity = "all"
%matplotlib inline

[20] from google.colab import drive
drive.mount('/content/drive')

Mounted at /content/drive
```

6 сек. выполнено в 10:50

```
[1] ls
drive/ sample_data/

[4] mkdir pyspark2024

[5] ls
drive/ pyspark2024/ sample_data/

[6] cd pyspark2024
/content/pyspark2024

[11] !unzip "/content/drive/MyDrive/endomondoHR.json.zip"
Archive: /content/drive/MyDrive/endomondoHR.json.zip
inflating: endomondoHR.json

[12] ls
```

6 сек. выполнено в 10:50

The screenshot shows a Google Colab notebook titled "advanced_pyspark_for_exploratory_data_analysis_student.ipynb". The interface includes a top bar with navigation options like "Файл", "Изменить", "Вид", "Вставка", "Среда выполнения", "Инструменты", and "Справка". Below this is a search bar and a "Команды" (Commands) section. The main code cell is titled "endomondoHR.json" and contains the following Python code:

```
[13] ! pip install pyspark

Requirement already satisfied: pyspark in /usr/local/lib/python3.11/dist-packages (3.5.5)
Requirement already satisfied: py4j==0.10.9.7 in /usr/local/lib/python3.11/dist-packages (from pyspark) (0.10.9.7)

# Импорт модулей, связанных с PySpark.
import pyspark
from pyspark.rdd import RDD
from pyspark.sql import Row
from pyspark.sql import DataFrame
from pyspark.sql import SparkSession
from pyspark.sql import SQLContext
from pyspark.sql import functions
from pyspark.sql.functions import lit, desc, col, size, array_contains,
, isnan, udf, hour, array_min, array_max, countDistinct
from pyspark.sql.types import *

MAX_MEMORY = '12G'
# Инициализировать сеанс Spark.
conf = pyspark.SparkConf().setMaster("local[*]") \
    .set("spark.executor.heartbeatInterval", 10000) \
```

The code cell has a status of "6 сек." (6 seconds) and "выполнено в 10:50" (completed at 10:50). The bottom of the interface shows a system tray with weather information (19°C, В осн. облачно), a search bar, and various application icons.

The screenshot shows the same Google Colab notebook, but with the next code cell executed. The code continues the initialization of the Spark session and loads data from a JSON file:

```
MAX_MEMORY = '12G'
# Инициализировать сеанс Spark.
conf = pyspark.SparkConf().setMaster("local[*]") \
    .set("spark.executor.heartbeatInterval", 10000) \
    .set("spark.network.timeout", 10000) \
    .set("spark.core.connection.ack.wait.timeout", "3600") \
    .set("spark.executor.memory", MAX_MEMORY) \
    .set("spark.driver.memory", MAX_MEMORY)

def init_spark():
    spark = SparkSession \
        .builder \
        .appName("Pyspark guide") \
        .config(conf=conf) \
        .getOrCreate()
    return spark

spark = init_spark()
filename_data = 'endomondoHR.json' # Загрузите данные в текущий каталог Colab.
# filename_data = 'data/endomondoHR.json' # Загрузите данные в каталог data/ при работе в Docker.
# Загрузите основной набор данных в фрейм данных pyspark.
df = spark.read.json(filename_data, mode="DROPMALFORMED")
print('Data frame type: ' + str(type(df)))

Data frame type: <class 'pyspark.sql.dataframe.DataFrame'>
```

The code cell has a status of "6 сек." (6 seconds) and "выполнено в 10:50" (completed at 10:50). The bottom of the interface shows the same system tray with weather information, a search bar, and various application icons.

28 10 диссер

colab.research.google.com advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

2. Обзор набора данных

[Go back to table of contents](#)

Схема, столбцы и типы данных набора:

Набор данных содержит как столбцы с одним значением (*int*, *string*), так и столбцы, состоящие из списка массивов.

```
print('Обзор данных')
df.printSchema()
print('Обзор столбцов')
pd.DataFrame(df.dtypes, columns = ['Column Name', 'Data type'])
```

Обзор данных

```
root
|-- altitude: array (nullable = true)
|   |-- element: double (containsNull = true)
|   |-- gender: string (nullable = true)
|   |-- heart_rate: array (nullable = true)
|   |   |-- element: long (containsNull = true)
|   |-- id: long (nullable = true)
|   |-- latitude: array (nullable = true)
|   |   |-- element: double (containsNull = true)
|   |-- longitude: array (nullable = true)
```

6 сек. выполнено в 10:50

19°C В осн. облачно

Поиск

11:20 23.04.2025

28 10 диссер

colab.research.google.com advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

```
|   |-- element: long (containsNull = true)
|-- url: string (nullable = true)
|-- userId: long (nullable = true)
```

Обзор столбцов

	Column Name	Data type
0	altitude	array<double>
1	gender	string
2	heart_rate	array<bigint>
3	id	bigint
4	latitude	array<double>
5	longitude	array<double>
6	speed	array<double>
7	sport	string
8	timestamp	array<bigint>
9	url	string
10	userId	bigint

6 сек. выполнено в 10:50

19°C В осн. облачно

Поиск

11:20 23.04.2025

28 10 диссер colab.research.google.com advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

3. Обнаружение пропущенных значений и аномальных нулей.

[Go back to table of contents](#)

После предварительного просмотра столбцов первое, что должны проверить, — это наличие в наборе данных какого-либо пропущенного значения. — Для строковых столбцов проверяем наличие None и null.

- Для числовых столбцов проверяем наличие нулей и NaN.
- Для столбцов типа массив проверяем, содержит ли массив нули или NaN.

```
print('Обзор столбцов')
pd.DataFrame(df.dtypes, columns = ['Column Name', 'Data type'])
```

Обзор столбцов

	Column Name	Data type
0	altitude	array<double>
1	gender	string
2	heart_rate	array<bigint>
3	id	bigint

6 сек. выполнено в 10:50

19°C В осн. облачно 11:21 23.04.2025

28 10 диссер colab.research.google.com advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

Обзор столбцов

	Column Name	Data type
0	altitude	array<double>
1	gender	string
2	heart_rate	array<bigint>
3	id	bigint
4	latitude	array<double>
5	longitude	array<double>
6	speed	array<double>
7	sport	string
8	timestamp	array<bigint>
9	url	string
10	userId	bigint

```
[18] string_columns = ['gender', 'sport', 'url']
      numeric_columns = ['id', 'userId']
```

6 сек. выполнено в 10:50

19°C В осн. облачно 11:21 23.04.2025

28 10 диссер

colab.research.google.com advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

```
string_columns = ['gender', 'sport', 'url']
numeric_columns = ['id', 'userId']
array_columns = ['altitude', 'heart_rate', 'latitude', 'longitude', 'speed', 'timestamp']
missing_values = {}
for index, column in enumerate(df.columns):
    if column in string_columns: # проверить столбцы строк со значениями None и Null
        missing_count = df.filter(col(column).isNullSafe(None) | col(column).isNull()).count()
        missing_values.update({column: missing_count})
    if column in numeric_columns: # check zeroes, None, NaN
        missing_count = df.where(col(column).isin([0, None, np.nan])).count()
        missing_values.update({column: missing_count})
    if column in array_columns: # check zeros and NaN
        missing_count = df.filter(array_contains(df[column], 0) | array_contains(df[column], np.nan)).count()
        missing_values.update({column: missing_count})
missing_df = pd.DataFrame.from_dict(missing_values)
missing_df
```

	altitude	gender	heart_rate	id	latitude	longitude	speed	sport	timestamp	url	userId
0	40848	0	1280	0	113	113	7741	0	0	0	0

6 сек. выполнено в 10:50

19°C В осн. облачно

Поиск

11:21 23.04.2025

28 10 диссер

colab.research.google.com advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

```
# создаем новый столбец для подсчета количества временных меток, записанных для каждой строки/тренировки, с именем столбца «PerWorkoutRecordCount»
df = df.withColumn('PerWorkoutRecordCount', size(col('timestamp')))

# Эта часть написана как функция, которую можно будет использовать позже.
def user_activity_workout_summarize(df):
    user_count = format(df.select('userId').distinct().count(), 'd')
    workout_count = format(df.select('id').distinct().count(), 'd')
    activity_count = str(df.select('sport').distinct().count())
    sum_temp = df.agg(functions.sum('PerWorkoutRecordCount')).toPandas()
    total_records_count = format(sum_temp['sum(PerWorkoutRecordCount)'][0], 'd')
    columns = ['Users count', 'Activity types count', 'Workouts count', 'Total records count']
    data = [[user_count], [activity_count], [workout_count], [total_records_count]]
    sum_dict = {column: data[i] for i, column in enumerate(columns)}
    sum_df = pd.DataFrame.from_dict(sum_dict)[columns]
    gender_user_count = df.select('gender', 'userId').distinct().groupBy('gender').count().toPandas()
    gender_activities_count = df.groupBy('gender').count().toPandas()
    gender_user_activity_count = gender_user_count.join(
        gender_activities_count.set_index('gender'), on='gender',
        how='inner', lsuffix='_gu'
    )
    gender_user_activity_count.columns = ['Gender', '# of users', 'Activities (workouts) count']

    return sum_df, gender_user_activity_count
```

6 сек. выполнено в 10:50

19°C В осн. облачно

Поиск

11:21 23.04.2025

2810 диссер

colab.research.google.comadvanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

ФайлИзменитьВидВставкаСреда выполненияИнструментыСправка

КомандыКодТекст

6 мин.

[19]gender_user_activity_count.columns = ['Gender', '# of users', 'Activities (workouts) count']

return sum_df, gender_user_activity_count

sum_dfs= user_activity_workout_summarize(df)
print('\nОбщая сводка набора данных о пользователях, тренировках и количестве записей (предварительная фильтрация):')
sum_dfs[0]

Общая сводка набора данных о пользователях, тренировках и количестве записей (предварительная фильтрация):
Users countActivity types countWorkouts countTotal records count
01,10449253,020111,541,956

Как мы видим, общее количество записей для этого набора данных превышает 111 миллионов записей. Это действительно большой размер

1 мин.

[20]print('Количество тренировок с менее чем 50 записями и статистической сводкой:')
removed_df = df.select('PerWorkoutRecordCount').where(df.PerWorkoutRecordCount < 50) \
.toPandas().describe().astype(int)
removed_df.rename(columns = {'PerWorkoutRecordCount': 'PerWorkoutRecordCount <50'}, inplace=True)
removed_df.T

6 сек.выполнено в 10:50

19°CВ осн. облачноПоиск11:2223.04.2025

2810 диссер

colab.research.google.comadvanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

ФайлИзменитьВидВставкаСреда выполненияИнструментыСправка

КомандыКодТекст

Количество тренировок с менее чем 50 записями и статистической сводкой:

countmeanstdmin25%50%75%max
PerWorkoutRecordCount <5055412314111223649

4. Ленивая оценка PySpark

[Go back to table of contents](#)

Здесь мы начнем знакомиться с некоторыми функциями продвинутого Spark: **Ленивая оценка**.

Ленивая оценка расширяет возможности Apache Spark за счет сокращения времени выполнения операций RDD. Он поддерживает граф происхождения для запоминания операций над RDD. мы можем просто помнить, что вся обработка в PySpark – это абстракция. Когда мы хотим вернуть результаты, мы на самом деле сообщаем Spark, какой конечный ответ вас интересует, и он определяет лучший способ получить его. В результате это оптимизирует производительность и обеспечивает отказоустойчивость. Чтобы увидеть результат, нам нужно вызвать `Spark.collect()`.

Обычно можем показать результаты с помощью синтаксиса: `df.take(k)` или `df.limit(k)`, чтобы получить результаты с k-строкой.

Когда k становится большим числом, выполнение этих двух способов, описанных выше, занимает много времени. Потому что

6 сек.выполнено в 10:50

19°CВ осн. облачноПоиск11:2223.04.2025

28 10 диссер

colab.research.google.com advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

ОЗУ Диск

```
[21] ranked_sport_users_df = df.select(df.sport, df.userId) \
    .distinct() \
    .groupBy(df.sport) \
    .count() \
    .orderBy("count", ascending=False)

# Топ-5 типов тренировок
highest_sport_users_df = ranked_sport_users_df.limit(5).toPandas()
# Переименование имени столбца: 'count' -> Users count Количество пользователей.
highest_sport_users_df.rename(columns = {'count':'Users count'}, inplace = True)
# Подсчет общего количества пользователей. Результат потребуется для подсчета процентного соотношения.
total_sports_users = ranked_sport_users_df.groupBy().sum().collect()[0][0]

ranked_sport_users_df.collect()[:5]

[Row(sport='run', count=865),
 Row(sport='bike', count=794),
 Row(sport='mountain bike', count=336),
 Row(sport='bike (transport)', count=252),
 Row(sport='walk', count=209)]
```

5. Исследовательский анализ данных

6 сек. выполнено в 10:50

19°C В осн. облачно

Поиск

11:22 23.04.2025

28 10 диссер

colab.research.google.com advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

ОЗУ Диск

```
# Calculate the percentage for the top 5 sports
highest_sport_users_df_renamed = highest_sport_users_df.copy()
highest_sport_users_df_renamed['percentage'] = highest_sport_users_df['Users count'] / total_sports_users * 100

# Creating the 'others' group
others = {
    'sport': 'others',
    'Users count': total_sports_users - sum(highest_sport_users_df_renamed['Users count']),
    'percentage': 100 - sum(highest_sport_users_df_renamed['percentage'])
}

# Convert 'others' into a DataFrame
others_df = pd.DataFrame([others])

# Use pd.concat() to append 'others' to the original DataFrame
highest_sport_users_df_renamed = pd.concat([highest_sport_users_df_renamed, others_df], ignore_index=True)

print('Топ-5 видов спорта, в которых участвует больше всего пользователей:')
print(highest_sport_users_df_renamed)

# Plotting
fig, axs = plt.subplots(nrows=1, ncols=2, figsize=plt.figaspect(0.35))

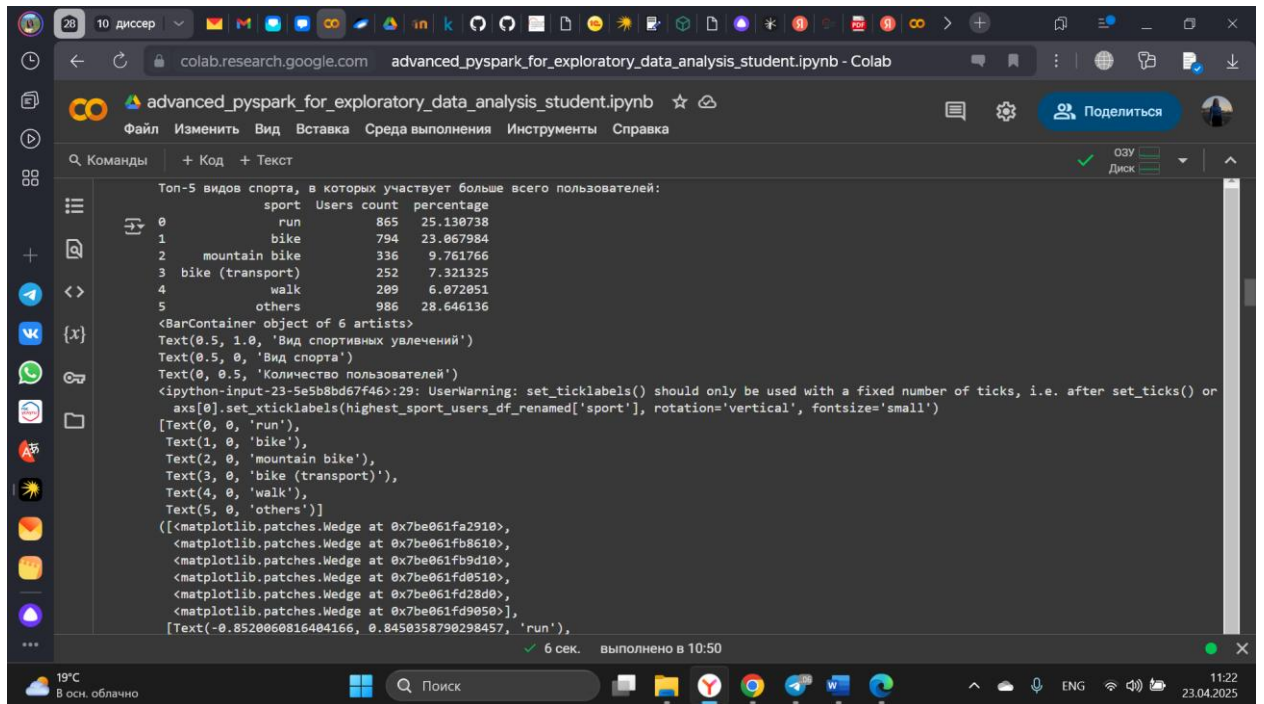
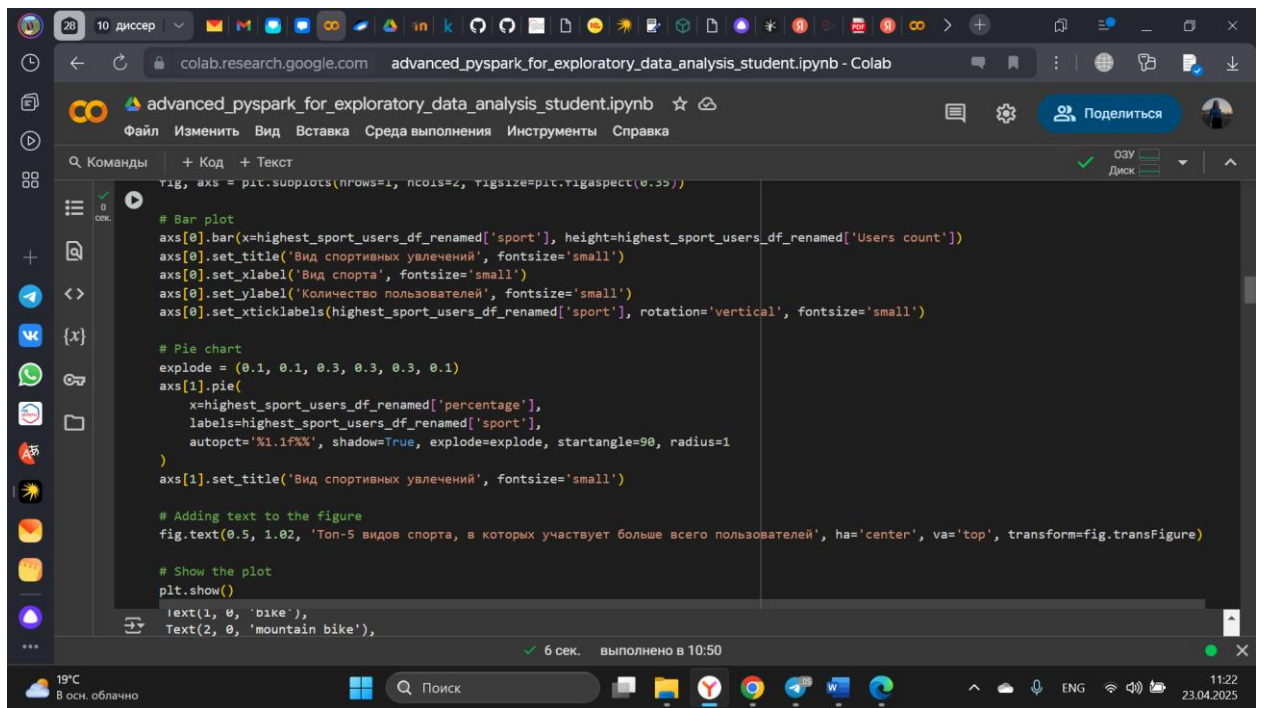
# Bar plot
```

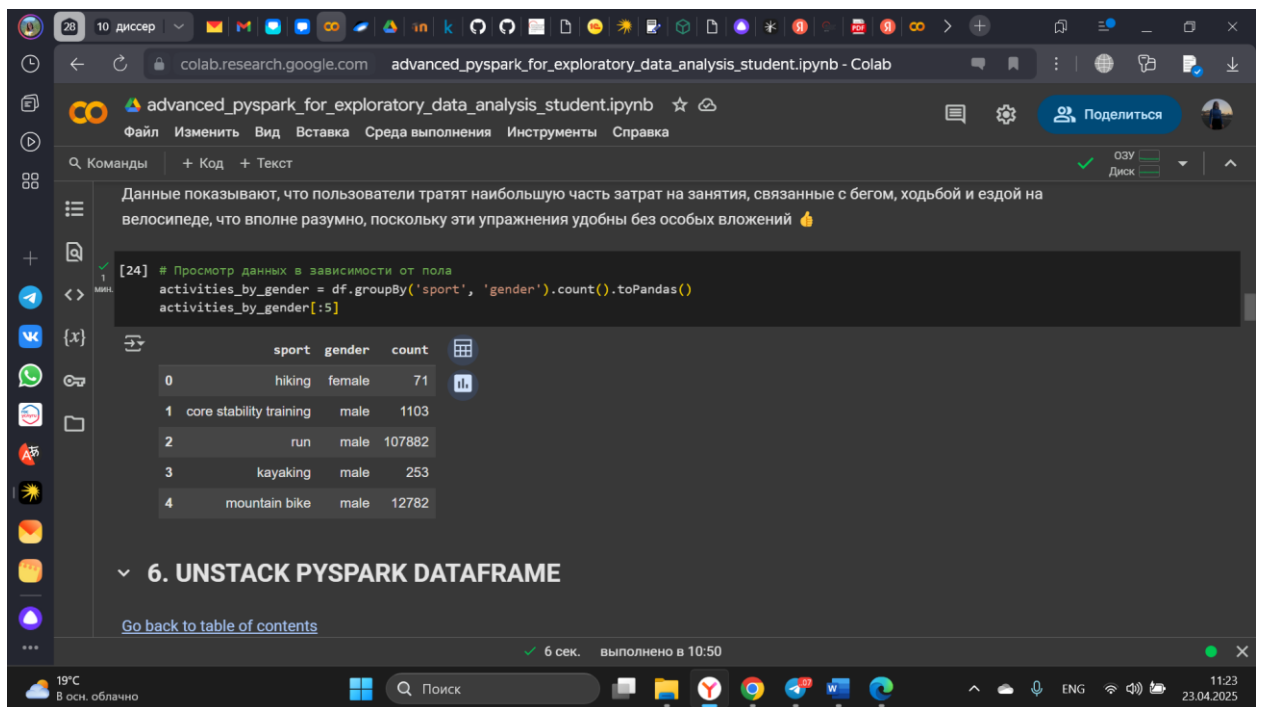
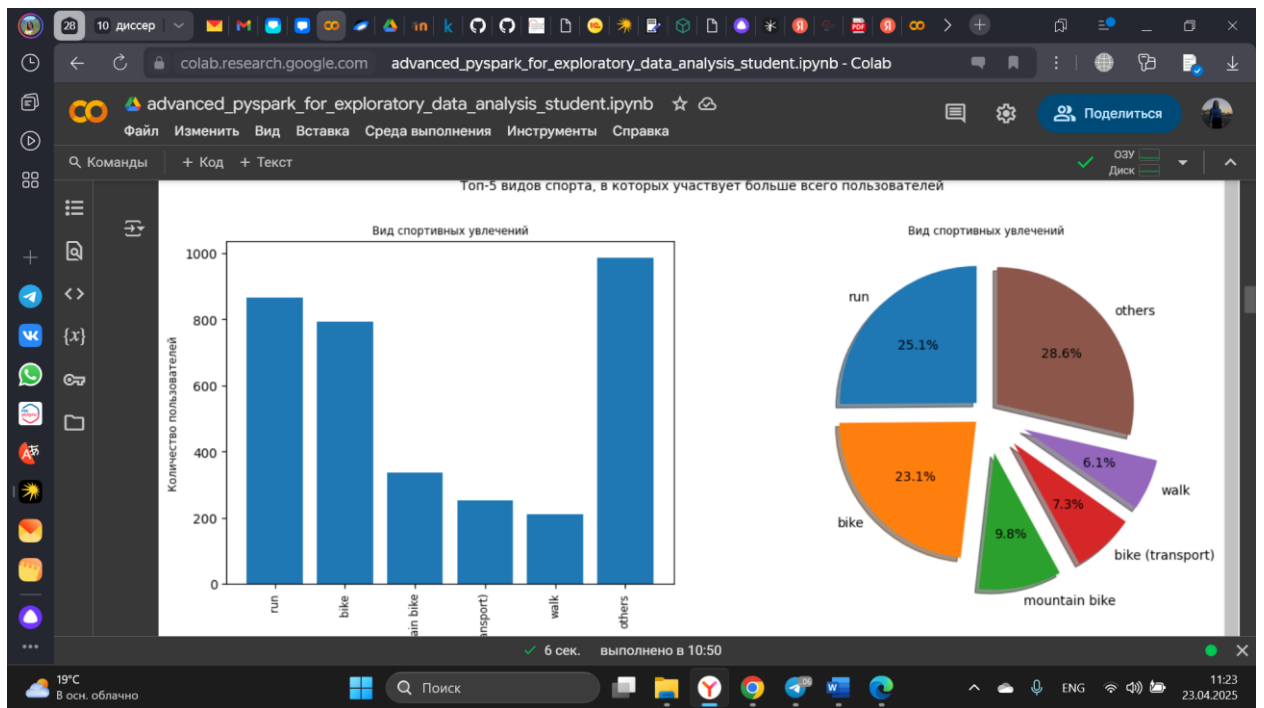
6 сек. выполнено в 10:50

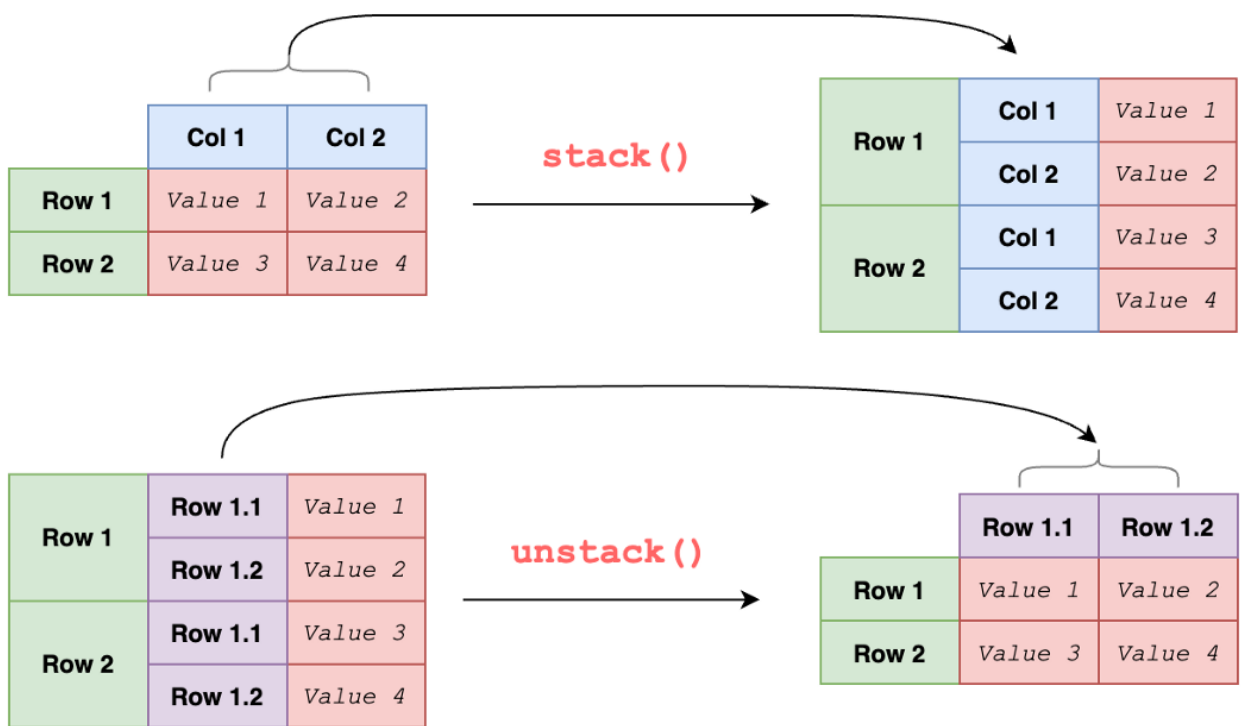
19°C В осн. облачно

Поиск

11:22 23.04.2025







advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

Чтобы изменить форму таблицы в PySpark, мы используем

```
df.unstack()
```

Относительно к нашим данным получим следующее разбиение:

	sport	gender	count
0	treadmill walking	male	2
1	step counter	male	2
2	swimming	unknown	8
3	fitness walking	male	228
4	circuit training	female	22
...	...	...	...
85	run	unknown	660
86	horseback riding	female	8

→

	sport	count	female	male	unknown
0	aerobics	3	43	0	0
1	badminton	0	17	0	0
2	basketball	0	14	0	0
3	beach volleyball	0	2	0	0
4	bike	4172	92966	863	0
5	bike (transport)	414	10030	1	0
6	circuit training	22	196	0	0

6 сек. выполнено в 10:50

19°C В осн. облачно

Поиск

11:24 23.04.2025

28 10 диссер

colab.research.google.com advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

ОЗУ Диск

```
total_activities = ranked_sport_users_df.count()
print(f'Всего: {total_activities} активности в зависимости от пола:')
# Добавление информации о занятиях в зависимости от пола
activities_by_gender = df.groupBy('sport', 'gender').count().toPandas()
# Визуализация
fig = plt.figure(figsize=(12, 25))
grid_size = (1,1);
ax = plt.subplot2grid(grid_size, (0,0), colspan=1, rowspan=1)
plot = activities_by_gender.groupby(['sport', 'gender']).agg(np.mean).groupby(level=0).apply(
    lambda x: 100 * x / x.sum()).unstack().plot(kind='barh', stacked=True, width=1 ## APPLY UNSTACK TO RESHAPE DATA
    , edgecolor='black', ax=ax, title='Список всех занятий в зависимости от пола')

ylabel = plt.ylabel('Sport (Activity)');
xlabel = plt.xlabel('Процент участия по гендерному признаку');
legend = plt.legend(
    sorted(activities_by_gender['gender'].unique()), loc='center left', bbox_to_anchor=(1.0, 0.5)
)

param_update = plt.rcParams.update({'font.size': 16});
ax = plt.gca()
formatter = ax.xaxis.set_major_formatter(mtick.PercentFormatter());
a = fig.tight_layout()
plt.show()
```

(kite surfing, kite surfing)

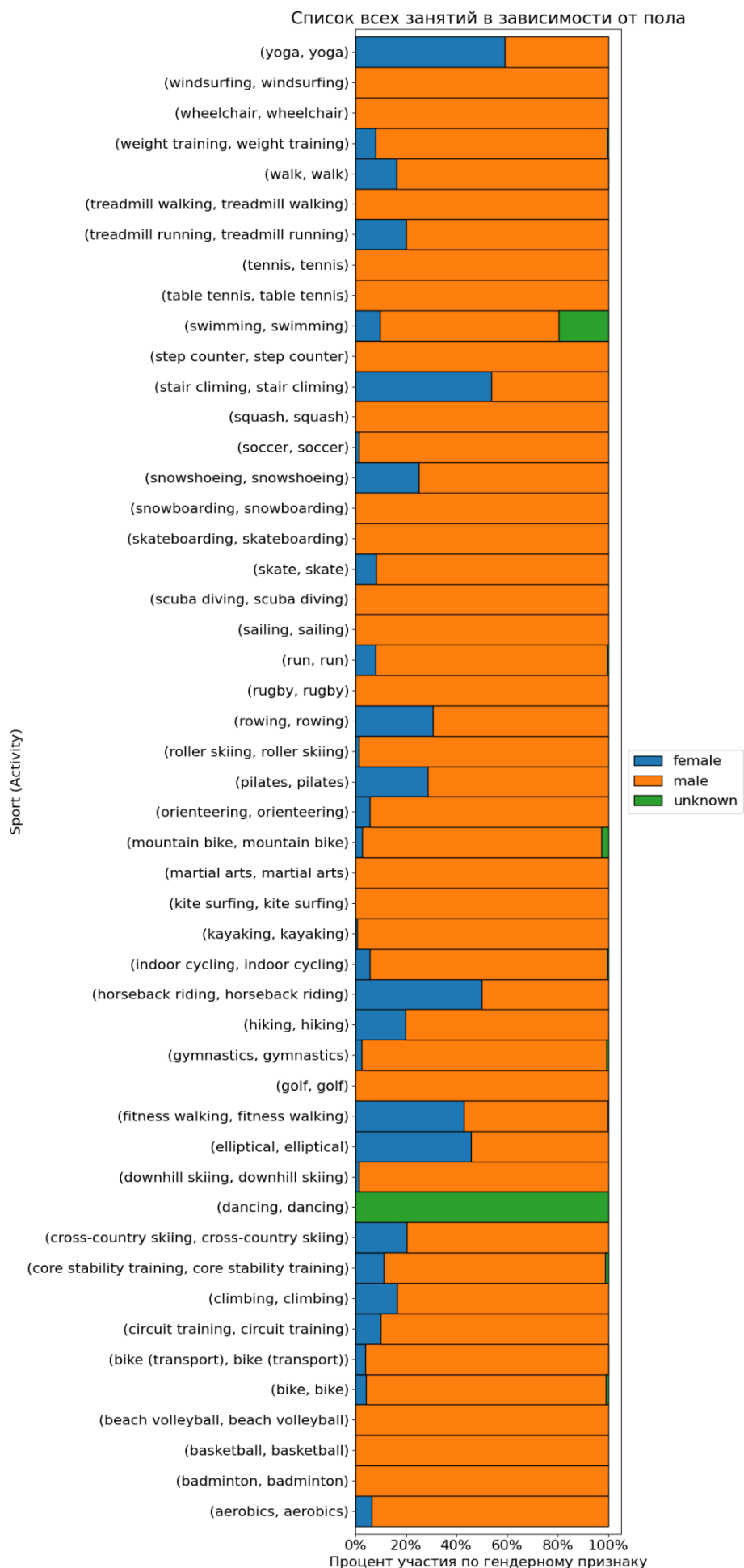
(kayaking, kayaking)

6 сек. выполнено в 10:50

19°C В осн. облачно

Поиск

11:24 23.04.2025



28 10 диссер

colab.research.google.com advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

Давайте посмотрим на верхнее парето из 5 видов спорта, в которых больше всего участников.

```
# Pivot the data by gender and sport
activities_by_gender_df = activities_by_gender.pivot_table(
    index="sport", columns="gender", values="count", fill_value=0) \
    .reset_index().rename_axis(None, axis=1)

# Add total count and percentage
activities_by_gender_df['total'] = activities_by_gender_df['male'] + activities_by_gender_df['female'] + activities_by_gender_df['unknown']
activities_by_gender_df['percentage'] = activities_by_gender_df['total'] / sum(activities_by_gender_df['total']) * 100

# Get top 5 activities by percentage
top_activities_by_gender_df = activities_by_gender_df.sort_values(by='percentage', ascending=False).head(5)

# Create the 'others' group
others = {'sport': 'others'}
for column in ['female', 'male', 'unknown', 'total', 'percentage']:
    value = sum(activities_by_gender_df[column]) - sum(top_activities_by_gender_df[column])
    others.update({column: value})

# Convert 'others' to a DataFrame
others_df = pd.DataFrame([others])
```

6 сек. выполнено в 10:50

19°C В осн. облачно

Поиск

11:24 23.04.2025

28 10 диссер

colab.research.google.com advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

```
# Use pd.concat to add 'others' row to the DataFrame
top_activities_by_gender_df = pd.concat([top_activities_by_gender_df, others_df], ignore_index=True)

# Sort by percentage again
top_activities_by_gender_df = top_activities_by_gender_df.sort_values(by='percentage', ascending=False)

# Display the final DataFrame
print(top_activities_by_gender_df)

# Plotting
fig, axs = plt.subplots(nrows=1, ncols=2, figsize=plt.figaspect(0.35))

# Bar plot for total count of activities
axs[0].bar(x=top_activities_by_gender_df['sport'], height=top_activities_by_gender_df['total'])
axs[0].set_title('Количество тренировок', fontsize='small')
axs[0].set_xlabel('Спорт', fontsize='small')
axs[0].set_ylabel('Количество тренировок (раз)', fontsize='small')
axs[0].set_xticklabels(top_activities_by_gender_df['sport'], rotation='vertical', fontsize='small')

# Pie chart for percentage distribution
explode = (0.1, 0.1, 0.3, 0.3, 0.3)
axs[1].pie(
    x=top_activities_by_gender_df['percentage'],
    labels=top_activities_by_gender_df['sport'],
    explode=explode)
```

6 сек. выполнено в 10:50

19°C В осн. облачно

Поиск

11:25 23.04.2025

```
explore = (0.1, 0.1, 0.3, 0.3, 0.3, 0.3)
axs[1].pie(
    x=top_activities_by_gender_df['percentage'],
    labels=top_activities_by_gender_df['sport'],
    autopct='%1.1f%%', shadow=True, explode=explode, radius=1
)
axs[1].set_title('Соотношение тренировок', fontsize='small')

# Add text to the figure
fig.text(0.5, 1.02, 'Топ-5 видов спорта', ha='center', va='top', transform=fig.transFigure)

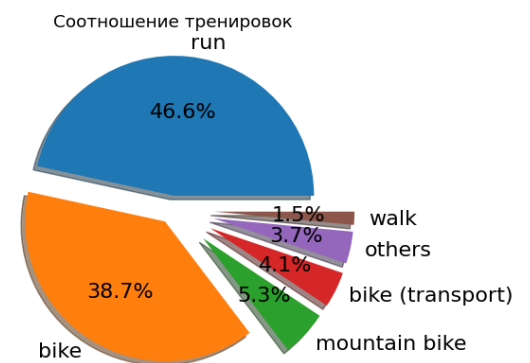
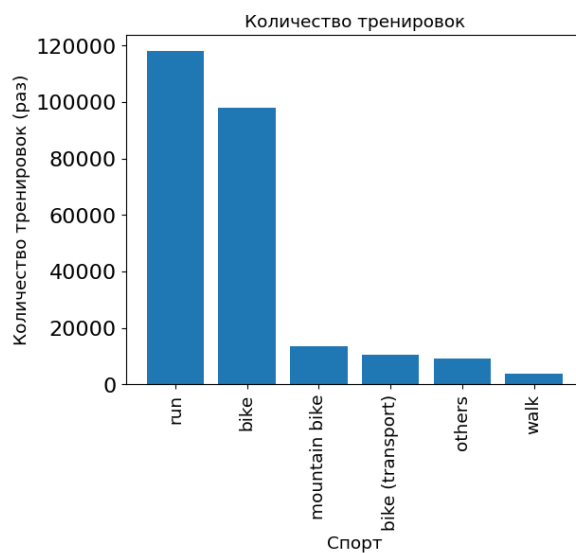
# Show the plot
plt.show()
```

	sport	female	male	unknown	total	percentage
0	run	9360.0	107882.0	660.0	117902.0	46.597897
1	bike	4172.0	92966.0	863.0	98001.0	38.732511
2	mountain bike	353.0	12782.0	375.0	13510.0	5.339499
3	bike (transport)	414.0	10030.0	1.0	10445.0	4.128132
5	others	1034.0	8255.0	42.0	9331.0	3.687851
4	walk	626.0	3204.0	1.0	3831.0	1.514110

<BarContainer object of 6 artists>
Text(0.5, 1.02, 'Количество тренировок')
Text(0.5, 0, 'Спорт')
Text(0, 0.5, 'Количество тренировок (раз)')
<iovtion-input-27-e994087bef71>:39: UserWarning: set ticklabels() should only be used with a fixed number of ticks. i.e. after set ticks() or

6 сек. выполнено в 10:50

Топ-5 видов спорта



28 10 диссер

colab.research.google.com advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

```
min_number_of_sports = 1

sport_df = df \
    .select(df.userId, df.gender, df.sport) \
    .distinct() \
    .groupBy(df.userId, df.gender) \
    .count()

user_more_sports_df = sport_df \
    .filter(sport_df["count"] > min_number_of_sports) \
    .orderBy("count", ascending = False) \
    .toPandas()

user_more_sports_df.rename(columns = {'count': 'Sports count'}, inplace = True)
user_more_sports_df.describe().astype(int).T
```

	count	mean	std	min	25%	50%	75%	max
userid	822	4860464	3953412	69	1609606	3730685	7554937	15481421
Sports count	822	3	2	2	2	3	5	16

+ Код + Текст

По итогам 822 человека занимались более чем 1 видом спорта. Среди них в среднем человек занимается примерно 3 видами

6 сек. выполнено в 10:50

19°C В осн. облачно

Поиск

11:25 23.04.2025

28 10 диссер

colab.research.google.com advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

```
plot = user_more_sports_df.boxplot(column='Sports count', by='gender', fontsize='small', figsize=(6,7))
```

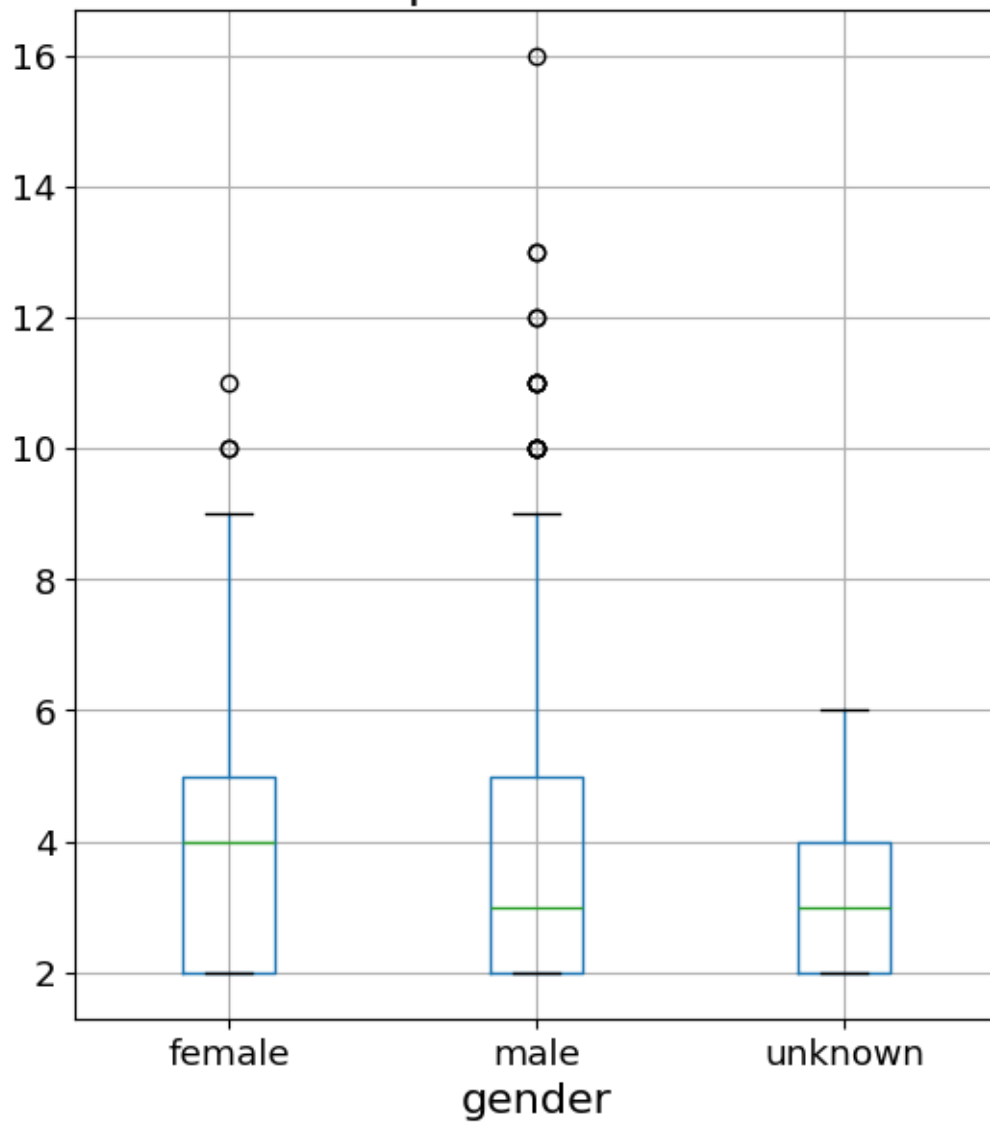
6 сек. выполнено в 10:50

19°C В осн. облачно

Поиск

11:25 23.04.2025

Boxplot grouped by gender
Sports count



28 10 диссер

colab.research.google.com advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

ОЗУ Диск

```
print('\nГрафик распределения тренировок по видам деятельности:')
plot_size_x, plot_size_y = 5, 5
figsize_x, figsize_y = plot_size_x * 4 + 3, plot_size_y * 13 + 1
figsize=(figsize_x, figsize_y)
fig = plt.figure(figsize=figsize) #
grid_size = (13,4)
ax = plt.subplot2grid(grid_size, (0,0), colspan=1, rowspan=1)
#fig, ax = plt.subplots()
PerWorkoutRecordCount_dist = df.select('PerWorkoutRecordCount', 'sport').toPandas().hist(
    column='PerWorkoutRecordCount', bins=10, sharex = False, grid=True
    , xlabelsize='small', ylabelsize='small', by='sport', ax = ax
    , layout = grid_size, figsize=figsize
)
a = fig.tight_layout()
title = fig.text(0.5, 1, 'Распределение количества рекордов за тренировку по видам спорта', ha='center'
    , fontsize='small', transform=fig.transFigure);
xlabel = fig.text(
    0.5, 0.01, '# рекорды/тренировки', va='bottom', ha='center', transform=fig.transFigure
)
ylabel = fig.text(0.01, 0.5, 'Частота (количество)', va='center', rotation='vertical');
```

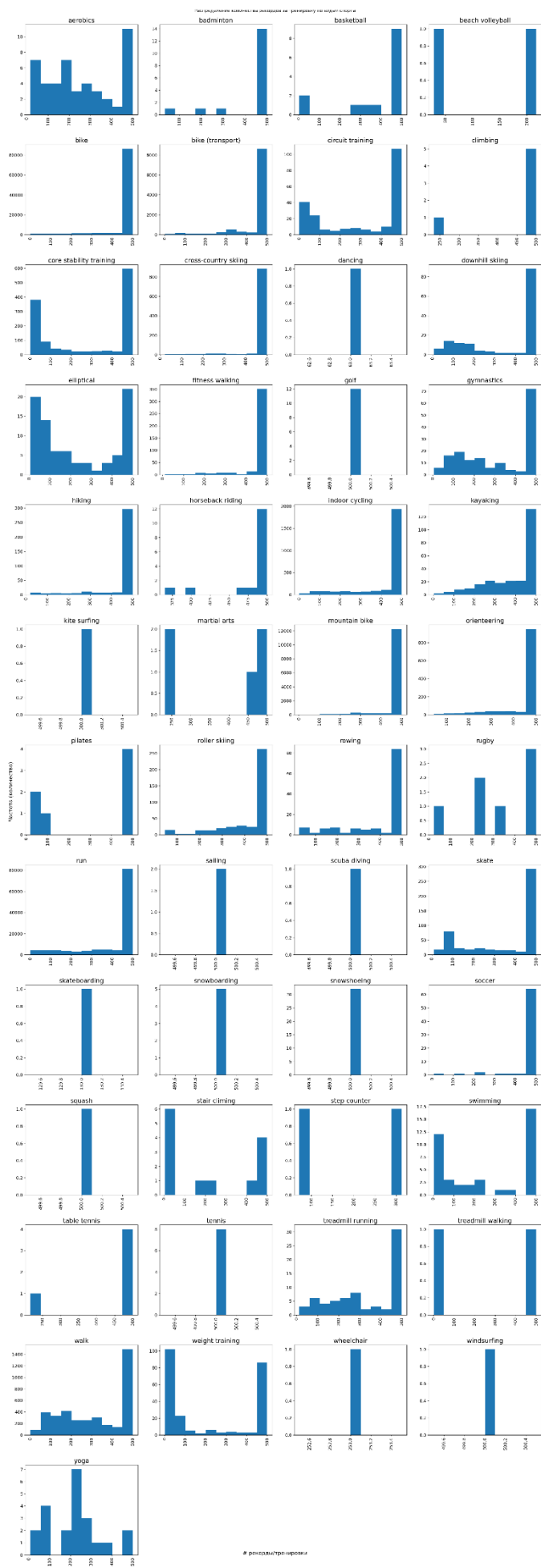
squash stair climbing step counter swimming

6 сек. выполнено в 10:50

19°C В осн. облачно

Поиск

11:26 23.04.2025



28 10 диссер

colab.research.google.com advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

0 сек.

```
[31] # Отфильтровать df с минимум 10 записями (поскольку мы предполагаем, что любой user_id с записью менее 10 не будет иметь смысла)
qualified_df = df \
    .select(df.sport, df.userId, df.gender) \
    .groupBy(df.sport, df.userId, df.gender) \
    .count()

qualified_df = qualified_df.filter(qualified_df["count"] >= 10) \
    .orderBy("count", ascending = False)
```

Чтобы изменить содержимое ячейки, дважды нажмите на нее (или выберите "Ввод")

Чтобы изменить содержимое ячейки, дважды нажмите на нее (или выберите "Ввод")

2 мин.

```
print("Количество пользователей, имеющих более 10 тренировок:")
qualified_pd_df = qualified_df.select("userId", "gender").distinct() \
    .groupBy(qualified_df.gender).count().toPandas()
qualified_pd_df.rename(columns={'count': 'Users count'}, inplace=True)
qualified_pd_df
qualified_users_count = sum(qualified_pd_df['Users count'])
total_users_count = df.select('userId').distinct().count()
qualified_percentage = round((qualified_users_count / total_users_count), 2) * 100
print('\nТаким образом, есть {} / {} пользователей, соответствующих 10 критериям исторических записей, что {:.2f}%'.format(qualified_users_count, total_users_count, qualified_percentage))
```

6 сек. выполнено в 10:50

19°C В осн. облачно

Поиск

11:26 23.04.2025

28 10 диссер

colab.research.google.com advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

2 мин.

```
qualified_percentage = round((qualified_users_count / total_users_count), 2) * 100
print('\nТаким образом, есть {} / {} пользователей, соответствующих 10 критериям исторических записей, что {:.2f}%'.format(qualified_users_count, total_users_count, qualified_percentage))
```

Количество пользователей, имеющих более 10 тренировок:

gender	Users count
0 unknown	13
1 female	88
2 male	886

Таким образом, есть 987 / 1104 пользователей, соответствующих 10 критериям исторических записей, что 89.00%

Далее: [Посмотреть рекомендованные графики](#) [New interactive sheet](#)

7. Pyspark UDF

[Go back to table of contents](#)

Что такое UDF?

6 сек. выполнено в 10:50

19°C В осн. облачно

Поиск

11:26 23.04.2025

28 10 диссер

colab.research.google.com advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

ОЗУ Диск

Чтобы прикрепить эти 4 функции к SparkDataFrame, регистрируем их с помощью PYSPARK UDF.

```
function_to_udf = udf(function, Datatype())  
  
# Datatype() can be floattype(), TimestampType(), etc
```

[33] # ПОСМОТРИТЕ СНОВА КОЛОНКУ ВРЕМЕННОЙ МЕТКИ
df.limit(3).toPandas()

	altitude	gender	heart_rate	id	latitude	longitude	speed	sport	timestamp
0	[41.6, 40.6, 40.6, 38.4, 37.0, 34.0, 34.0, 34....	male	[100, 111, 120, 119, 120, 116, 125, 128, 131, ...	396826535	[60.173348765820265, 60.173239801079035, 60.17... 60.17...	[24.64977040886879, 24.65014273300767, 24.6509... 24.6509...	[6.8652, 16.4736, 19.1988, 20.4804, 31.3956, 3... 3...	bike	[1408898746, 1408898754, 1408898765, 140889877... https://www.endomondo.com/users/11
1	[38.4, 39.0, 39.0, 38.2, 36.8, 36.8, 36.8, 35....	male	[100, 105, 111, 110, 108, 115, 126, 130, 132, ...	392337038	[60.173247596248984, 60.17320962622762, 60.172... 60.172...	[24.649855233728886, 24.65015547350049, 24.650... 24.650...	[9.0792, 13.284, 15.9336, 10.9476, 16.1676, 30... 30...	bike	[1408221682, 1408221687, 1408221699, 140822170... https://www.endomondo.com/users/11
2	[76.4, 73.2, 72.4, 73.0, 73.0, 70.8, 69.2, 69....	male	[99, 105, 113, 110, 109, 110, 108, 121, 116, 1...]	389643739	[60.17326201312244, 60.173205351457, 60.172969... 60.172969...	[24.64995657093823, 24.650197718292475, 24.651... 24.651...	[4.0464, 13.8204, 16.9776, 19.9404, 28.7856, 3... 3...	bike	[1407858459, 1407858466, 1407858478, 140785849... https://www.endomondo.com/users/11

6 сек. выполнено в 10:50

19°C В осн. облачно

Поиск

11:26 23.04.2025

28 10 диссер

colab.research.google.com advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

ОЗУ Диск

0	[41.6, 40.6, 40.6, 38.4, 37.0, 34.0, 34.0, 34....	male	[100, 111, 120, 119, 120, 116, 125, 128, 131, ...	396826535	[60.173348765820265, 60.173239801079035, 60.17... 60.17...	[24.64977040886879, 24.65014273300767, 24.6509... 24.6509...	[6.8652, 16.4736, 19.1988, 20.4804, 31.3956, 3... 3...	bike	[1408898746, 1408898754, 1408898765, 140889877... https://www.endomondo.com/users/11
1	[38.4, 39.0, 39.0, 38.2, 36.8, 36.8, 36.8, 35....	male	[100, 105, 111, 110, 108, 115, 126, 130, 132, ...	392337038	[60.173247596248984, 60.17320962622762, 60.172... 60.172...	[24.649855233728886, 24.65015547350049, 24.650... 24.650...	[9.0792, 13.284, 15.9336, 10.9476, 16.1676, 30... 30...	bike	[1408221682, 1408221687, 1408221699, 140822170... https://www.endomondo.com/users/11
2	[76.4, 73.2, 72.4, 73.0, 73.0, 70.8, 69.2, 69....	male	[99, 105, 113, 110, 109, 110, 108, 121, 116, 1...]	389643739	[60.17326201312244, 60.173205351457, 60.172969... 60.172969...	[24.64995657093823, 24.650197718292475, 24.651... 24.651...	[4.0464, 13.8204, 16.9776, 19.9404, 28.7856, 3... 3...	bike	[1407858459, 1407858466, 1407858478, 140785849... https://www.endomondo.com/users/11

6 сек. выполнено в 10:50

19°C В осн. облачно

Поиск

11:27 23.04.2025

28 10 диссер

colab.research.google.com advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

ОЗУ Диск

Мы создаем 4 вспомогательные функции для столбца 'timestamp', как описано выше, а затем преобразуем их в UDF.

```
# Преобразование столбца метки времени в Datetime.Datetime, чтобы позже использовать его для функции .withColumn.
def to_time(timestamp_list):
    # преобразовать в дату и время и минус 7 часов из-за разницы во временном окне Endomondo с временем utc в качестве описания набора данных
    return [datetime.fromtimestamp(t) - timedelta(hours=7) for t in timestamp_list]

# Регистрация вспомогательной функции to_time в структуре UDF pyspark
udf_to_time = udf(to_time, ArrayType(elementType=TimestampType()))

# Вспомогательная функция для получения продолжительности (в минутах) списка значений даты и времени, которая будет позже использована для функ
def get_duration(datetime_list):
    time_dif = max(datetime_list) - min(datetime_list)
    return time_dif.seconds/60

# Регистрация вспомогательной функции get_duration как пользовательской функции в pyspark.
udf_get_duration = udf(get_duration, FloatType())

# Вспомогательная функция для получения времени начала тренировки из списка даты и времени, которая будет позже использована для функции withCo
def get_start_time(datetime_list):
    return min(datetime_list)
```

6 сек. выполнено в 10:50

19°C В осн. облачно

11:27 23.04.2025

28 10 диссер

colab.research.google.com advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

ОЗУ Диск

```
for i in range(0, len(datetime_list)-1):
    interval = (datetime_list[i+1] - datetime_list[i]).seconds
    interval_list.append(interval)
return interval_list

# Регистрация вспомогательной функции get_interval как пользовательской функции в pyspark.
udf_get_interval = udf(get_interval, ArrayType(elementType=IntegerType()))

# Создание нового столбца date_time для преобразования метки времени в формат даты и времени Python для последующего использования.
df = df.withColumn('date_time',
    udf_to_time('timestamp'))

# Создание столбца «workout_start_time», чтобы получить время начала каждой тренировки/строки:
df = df.withColumn('workout_start_time', hour(udf_get_start_time('date_time')))

# Создание столбца продолжительности из только что созданного столбца date_time, используя функцию udf_get_duration, определенную выше.
df = df.withColumn('duration', udf_get_duration('date_time'))

# Создание столбца интервала из столбца date_time, используя функцию udf_get_interval, определенную выше.
df = df.withColumn('interval', udf_get_interval('date_time'))

print("Новые столбцы ('date_time', 'workout_start_time' in hour\
, 'duration' in minutes & 'interval' in seconds)\n, first 5 rows:")
df.select('timestamp', 'date_time', 'workout_start_time', 'duration', 'interval').limit(5).toPandas()
```

6 сек. выполнено в 10:50

19°C В осн. облачно

11:27 23.04.2025

28 10 диссер

colab.research.google.com advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

ОЗУ Диск

Новые столбцы (date_time, workout_start_time in hour, duration in minutes & interval in seconds), first 5 rows:

	timestamp	date_time	workout_start_time	duration	interval
0	[1408898746, 1408898754, 1408898765, 1408898777...]	[2014-08-24 09:45:46, 2014-08-24 09:45:54, 201...	9	126.483330	[8, 11, 13, 16, 6, 23, 16, 23, 29, 23, 24, 24, ...]
1	[1408221682, 1408221687, 1408221699, 140822170...]	[2014-08-16 13:41:22, 2014-08-16 13:41:27, 201...	13	74.000000	[5, 12, 8, 4, 5, 6, 4, 4, 5, 13, 7, 17, 4, 10, ...]
2	[1407858459, 1407858466, 1407858478, 140785849...]	[2014-08-12 08:47:39, 2014-08-12 08:47:46, 201...	8	112.483330	[7, 12, 12, 16, 6, 7, 16, 11, 19, 13, 17, 11, ...]
3	[1407432042, 1407432048, 1407432056, 140743206...]	[2014-08-07 10:20:42, 2014-08-07 10:20:48, 201...	10	75.316666	[6, 8, 9, 4, 13, 4, 3, 4, 13, 10, 13, 13, 7, 1...]
4	[1406909434, 1406909436, 1406909437, 140690943...]	[2014-08-01 09:10:34, 2014-08-01 09:10:36, 201...	9	22.616667	[2, 1, 1, 3, 2, 1, 4, 2, 1, 2, 1, 1, 2, 4, 4, ...]

Теперь посмотрим на продолжительность каждой тренировки (в минутах).

```
[35] print('\nСтатистика длительности (в минутах):')
df.select('duration').toPandas().describe().T
```

6 сек. выполнено в 10:50

19°C В осн. облачно

Поиск

11:27 23.04.2025

28 10 диссер

colab.research.google.com advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

ОЗУ Диск

Теперь посмотрим на продолжительность каждой тренировки (в минутах).

```
[35] print('\nСтатистика длительности (в минутах):')
df.select('duration').toPandas().describe().T
```

Статистика длительности (в минутах):

	count	mean	std	min	25%	50%	75%	max
duration	253020.0	84.486427	98.089287	0.0	32.166668	58.916668	102.73333	1439.966675

Из статистики столбца duration видно, что продолжительность тренировки может длиться от 0 минут до 1 полного дня (1440 минут = 24 часа). Продолжительность 0 может быть для тренировок, которые имеют только одну запись, поэтому минимальная и максимальная отметки времени будут одинаковыми.

Теперь пришло время построения графика продолжительности:

```
[36] print('\nПостроение графика распределения продолжительности по видам спорта:')
plot_size_x, plot_size_y = 5, 5
figsize_x, figsize_y = plot_size_x * 4 + 3, plot_size_y * 13 + 1
```

6 сек. выполнено в 10:50

19°C В осн. облачно

Поиск

11:27 23.04.2025

28 10 диссер

colab.research.google.com advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

ОЗУ Диск

```
print('\nПостроение графика распределения продолжительности по видам спорта:')
plot_size_x, plot_size_y = 5, 5
figsize_x, figsize_y = plot_size_x * 4 + 3, plot_size_y * 13 + 1
figsize = (figsize_x, figsize_y)
fig = plt.figure(figsize=figsize) #
grid_size = (13,4)
ax = plt.subplot2grid(grid_size, (0,0), colspan=1, rowspan=1)

duration_dist = df.select('duration', 'sport').toPandas().hist(
    column='duration', by='sport', bins=15, sharex = False, grid=True
    , xlabelsize='small', ylabelsize='small' , ax = ax
    , layout = grid_size, figsize=figsize
)

a = fig.tight_layout()
title = fig.text(0.5, 1, 'Распределение продолжительности тренировок по видам спорта'
    , ha='center', va='center', transform=fig.transFigure
)

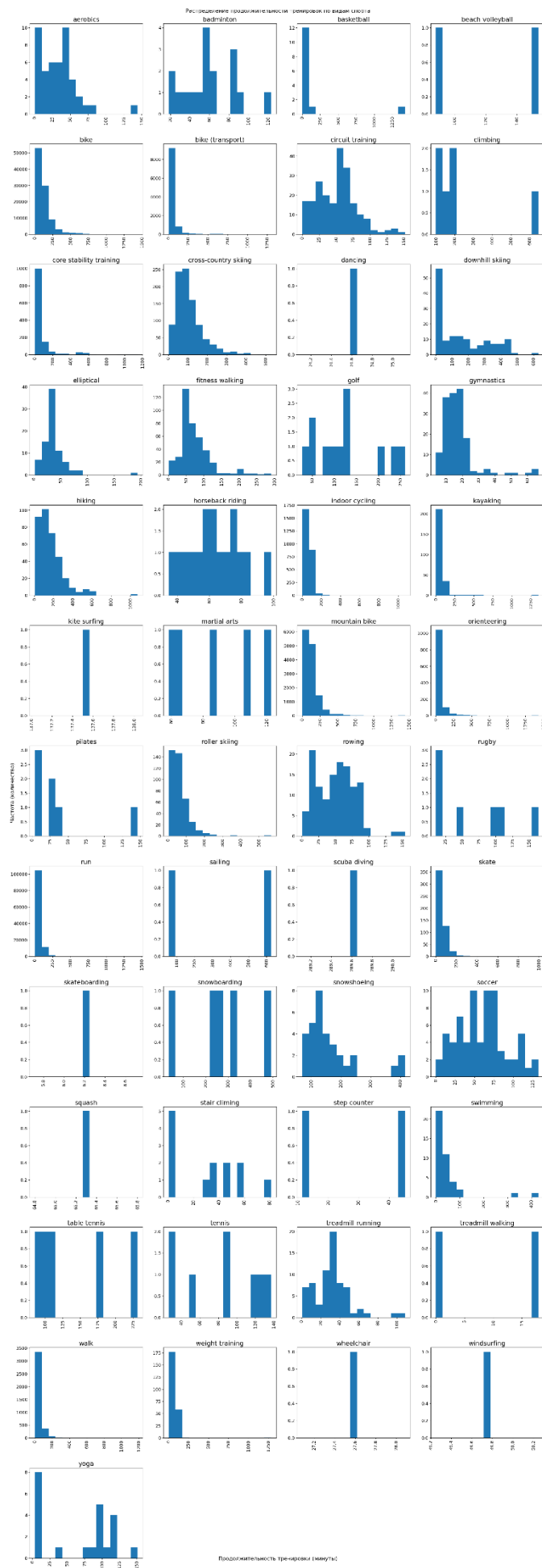
xlabel = fig.text(0.5, 0.01, 'Продолжительность тренировки (минуты)'
    , ha='center', va='center', transform=fig.transFigure)
ylabel = fig.text(0.01, 0.5, 'Частота (количество)', va='center', rotation='vertical');
```

6 сек. выполнено в 10:50

19°C В осн. облачно

Поиск

11:27 23.04.2025



28 10 диссер colab.research.google.com advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

ОЗУ Диск

Судя по сводке продолжительности и графику распределения, большинство занятий происходит в течение 1–2 часов, только несколько видов спорта и несколько случаев каждого типа происходят в течение более длительных периодов, таких как поездка на горном велосипеде, пеший туризм, парусный спорт и т. д. *

8.Преобразование объектов строк в устойчивый распределенный набор данных Spark (RDD)

[Go back to table of contents](#)

На этом графике мы попрактикуемся в преобразовании объекта строки в формат RDD в PySpark с помощью:

```
rdd = df.rdd.map(tuple)
or
rdd = df.rdd.map(list)
```

Расширенный формат RDD: каждый набор данных разделен на логические части, которые можно легко вычислить на разных узлах кластера. Они могут работать параллельно и являются отказоустойчивыми, поэтому процесс является стабильным и очень

6 сек. выполнено в 10:50

19°C В осн. облачно 11:28 23.04.2025

28 10 диссер colab.research.google.com advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

ОЗУ Диск

9 мин.

```
# Вспомогательная функция для расчета статистики имени столбца из кортежа x (спорт, список записей столбца) tuple x of (sport, records list of
#, статистика для расчета также предоставляется в качестве входных данных
def calculate_stats(x, column_name, stat_list):
    sport, records_list = x
    stat_dict = {'sport': sport}
    if 'min' in stat_list:
        min_stat = min(records_list)
        stat_dict.update({'min ' + column_name : min_stat})
    if 'max' in stat_list:
        max_stat = max(records_list)
        stat_dict.update({'max ' + column_name: max_stat})
    if 'mean' in stat_list:
        average_stat = stats.mean(records_list)
        stat_dict.update({'mean ' + column_name: average_stat})
    if 'stdev' in stat_list:
        std_stat = stats.stdev(records_list)
        stat_dict.update({'stdev ' + column_name: std_stat})
    if '50th percentile' in stat_list:
        median_stat = stats.median(records_list)
        stat_dict.update({'50th percentile ' + column_name: median_stat})
    if '25th percentile' in stat_list:
        percentile_25th_stat = np.percentile(records_list, 25)
        stat_dict.update({'25th percentile ' + column_name: percentile_25th_stat})
    if '75th percentile' in stat_list:
```

6 сек. выполнено в 10:50

19°C В осн. облачно 11:28 23.04.2025


```
def extend(a, b):
    a.extend(b)
    return a

def retrieve_array_column_stat_df(df, column_name, stat_list):
    # Преобразование спорт и «column_name» в RDD, чтобы легко рассчитать статистику интервалов по видам спорта.
    sport_record_rdd = df.select('sport', column_name).rdd \
        .map(tuple).combineByKey(to_list, extend, extend).persist()

    # Вычислить статистику входного столбца, вызвав функцию CalcStats, определенную выше.
    record_statistic_df = pd.DataFrame(sport_record_rdd.map(
        lambda x: calculate_stats(x, column_name, stat_list)).collect()
    )

    # Установка правильного порядка столбцов данных.
    columns_order = ['sport'] + [stat + ' ' + column_name for stat in stat_list]
    # Изменение порядка столбцов
    return record_statistic_df[columns_order]

stat_list = ['min', '25th percentile', 'mean', '50th percentile',
             '75th percentile', '95th percentile', 'max', 'stdev']
interval_statistic_df = retrieve_array_column_stat_df(df, column_name='interval', stat_list=stat_list)
print('\nПросмотр статистики интервалов в секундах (по виду спорта)')
interval_statistic_df
```

18	yoga	0	3.0	18.114438	7.0	19.00	70.00	1170	39.695865
19	step counter	1	7.0	8.877285	10.0	12.00	13.00	16	3.591450
20	downhill skiing	0	6.0	25.353402	16.0	26.00	63.00	5906	95.364198
21	fitness walking	0	5.0	9.265518	8.0	11.00	21.00	3403	19.103858
22	scuba diving	10	21.0	34.823647	33.0	43.00	62.00	123	16.067158
23	badminton	0	4.0	8.181660	7.0	10.00	17.00	1677	21.833185
24	weight training	0	3.0	21.347607	6.0	10.00	60.00	82806	595.522372
25	snowshoeing	1	9.0	19.690256	14.0	20.00	38.00	8197	106.664846
26	walk	0	3.0	7.912633	5.0	8.00	18.00	86399	194.586603
27	horseback riding	0	4.0	7.932153	7.0	10.00	18.00	291	7.298296
28	rugby	1	3.0	11.860455	5.0	12.00	36.00	1744	44.593851
29	wheelchair	1	4.0	6.567460	6.0	9.00	13.00	17	3.769163
30	windsurfing	3	3.0	5.981964	3.0	3.00	6.00	809	37.434249
31	run	0	4.0	8.023294	6.0	9.00	18.00	86399	130.336712

28 10 диссер

colab.research.google.com advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

ОЗУ Диск

```
import numpy as np
import matplotlib.pyplot as plt

print("\nОбобщенная статистика интервальных видов спорта:")

bar_columns = ['25th percentile interval', '50th percentile interval', '75th percentile interval', '95th percentile interval']
line_columns1 = ['min interval', 'mean interval']
line_columns2 = ['max interval', 'stdev interval']

interval_statistic_df = interval_statistic_df.sort_values(by='95th percentile interval', ascending=False)

figsize = (13, 59)
fig, axs = plt.subplots(nrows=7, figsize=figsize)

d = axs[0].set_title('Интервальная статистика по видам спорта', fontsize=18)

for i in range(7):
    interval_statistic_sub_df = interval_statistic_df.iloc[i*7:i*7+7]

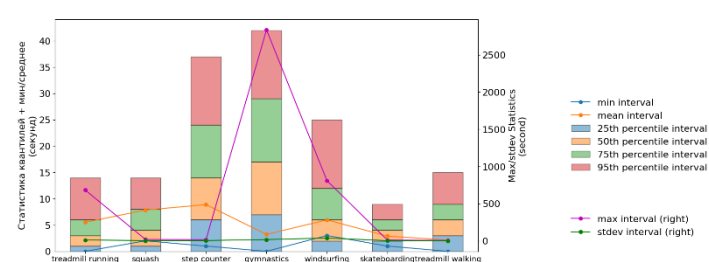
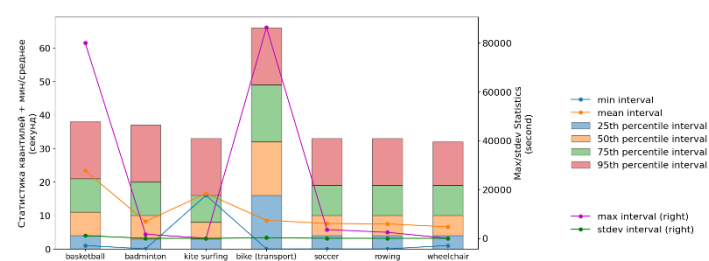
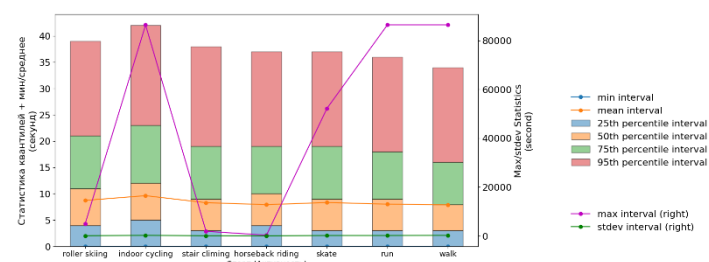
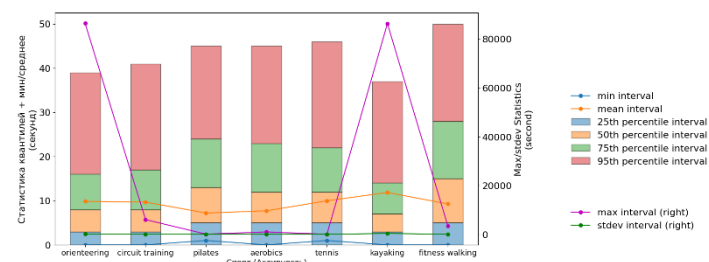
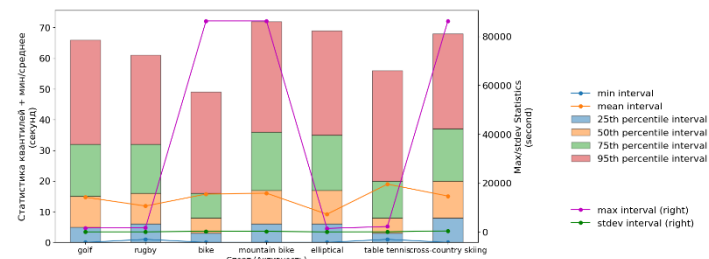
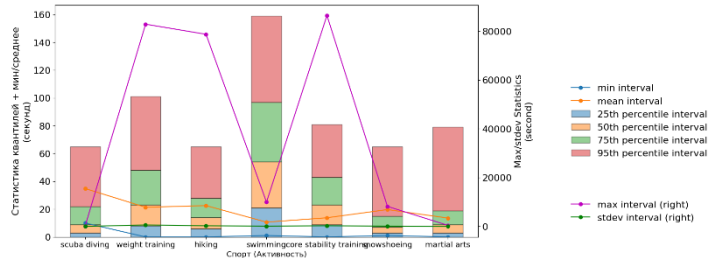
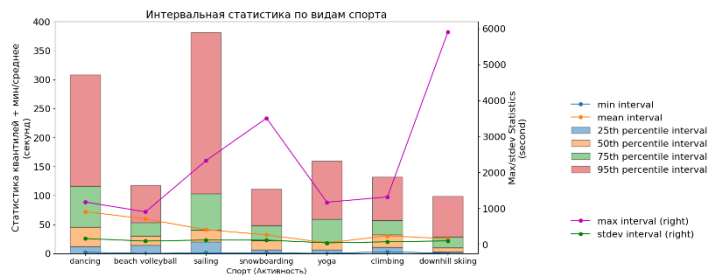
    # Plot the bar chart for the quantiles
    plot1 = interval_statistic_sub_df[['sport'] + bar_columns].groupby(['sport']).agg(np.mean).plot(
        kind='bar', stacked=True, grid=False, alpha=0.5, edgecolor='black', ax=axs[i]
    )
```

6 сек. выполнено в 10:50

19°C В осн. облачно

Поиск

11:29 23.04.2025



28 10 диссер

colab.research.google.com advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

ОЗУ Диск

```
[39] # Получите таблицу пола, вида спорта и времени начала тренировки для построения графика.
start_time_df = df.select('gender', 'sport', 'workout_start_time').toPandas()

activities = start_time_df['sport'].unique()
plot_size_x, plot_size_y = 5, 5
figsize_x, figsize_y = (plot_size_x + 0.5) * 4 + 3, (plot_size_y + 1) * 13 + 1

nrows, ncols = 13, 4
a = fig.subplots_adjust(hspace = 1, wspace = 1)
fig, axes = plt.subplots(nrows=nrows, ncols=ncols, figsize=(figsize_x, figsize_y))
print('\nГрафик распределения времени начала тренировки по видам спорта с разбивкой по полу:')
a = plt.setp(axes, xticks=[0, 4, 8, 12, 16, 20])
for index, sport in enumerate(activities):
    row_index, col_index = divmod(index, ncols)
    male_start_time_list = start_time_df[(start_time_df.sport == sport) &
                                          (start_time_df.gender == 'male')]['workout_start_time']
    female_start_time_list = start_time_df[(start_time_df.sport == sport) &
                                           (start_time_df.gender == 'female')]['workout_start_time']
    unknown_start_time_list = start_time_df[(start_time_df.sport == sport) &
                                             (start_time_df.gender == 'unknown')]['workout_start_time']
    if len(male_start_time_list) > 0:
```

6 сек. выполнено в 10:50

19°C В осн. облачно

Поиск

11:29 23.04.2025



28 10 диссер colab.research.google.com advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

ОЗУ Диск

```
[41] stat_list = ['min', '25th percentile', 'mean', '95th percentile', 'max', 'stdev']
heart_rate_statistic_df = retrieve_array_column_stat_df(df, column_name='heart_rate', stat_list=stat_list)
```

Из-за огромного количества пользователей и количества тренировок мы случайным образом выбрали до x количества пользователей каждого пола (например, 5) и до y тренировок по каждому типу активности (например, 10).

```
# Вспомогательная функция, которая помогает отбирать данные
def sampling_data(max_users_per_gender, max_workouts_per_sport):
    ...
    max_users_per_gender: максимальное количество пользователей, выбираемых случайным образом для каждого пола.
    max_workouts_per_sport: максимальное количество занятий, которые можно выбрать для каждого вида спорта.
    (виды спорта, существующие у выбранных пользователей)
    ...
    # Получение уникального списка идентификаторов пользователя и пола для выборки.
    users_genders = df.select('userId', 'gender').distinct().toPandas()
    # Выбор до трех идентификаторов пользователей каждого пола из уникального списка идентификаторов пользователей.
    random_x_users_per_gender = users_genders.groupby('gender')['userId'].apply(
        lambda s: s.sample(min(len(s), max_users_per_gender))
    )
    ...
    # Применение фильтра к pyspark dataframe для выборки.
    samples_by_gender = df.where(df.userId.isin(list(random_x_users_per_gender)))
```

6 сек. выполнено в 10:50

19°C В осн. облачно

Поиск

11:29 23.04.2025

28 10 диссер colab.research.google.com advanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

Файл Изменить Вид Вставка Среда выполнения Инструменты Справка

Команды + Код + Текст

ОЗУ Диск

```
print('\nОбзор выборочных данных (только строковые и числовые столбцы):')
pd_df.describe()
```

Обзор выборочных данных (только строковые и числовые столбцы):

	id	userId	PerWorkoutRecordCount	workout_start_time	duration
count	2.440000e+02	2.440000e+02	244.000000	244.000000	244.000000
mean	3.446910e+08	4.446380e+06	418.889344	12.795082	81.330666
std	1.607459e+08	3.574375e+06	156.452967	4.691185	91.859673
min	1.285100e+07	2.793170e+05	2.000000	0.000000	0.016667
25%	2.509808e+08	1.796601e+06	451.000000	10.000000	34.083332
50%	3.399247e+08	3.961631e+06	500.000000	13.000000	58.925001
75%	4.624037e+08	7.996903e+06	500.000000	16.000000	90.616667
max	6.504680e+08	1.384082e+07	500.000000	23.000000	866.000000

нормализуем время для всех тренировок, рассчитав продолжительность (в секундах) каждой записи временной метки из первой записи тренировки (первый элемент datetime списка в этой тренировке).

6 сек. выполнено в 10:50

19°C В осн. облачно

Поиск

11:29 23.04.2025

2810 диссер

colab.research.google.comadvanced_pyspark_for_exploratory_data_analysis_student.ipynb - Colab

advanced_pyspark_for_exploratory_data_analysis_student.ipynb

ФайлИзменитьВидВставкаСреда выполненияИнструментыСправка

КомандыКодТекст

ОЗУДиск

13 сек.

```
# Лямбда-функция для объединения списка списков в один большой список
flatten = lambda l: set([item for sublist in l for item in sublist])

normalized_datetime_list = []
for index, data_row in pd_df.iterrows():
    min_date_time = min(data_row['date_time'])
    normalized_datetime_list.append(
        [(date_time - min_date_time).seconds for date_time in data_row['date_time']]
    )

pd_df['normalized_date_time'] = normalized_datetime_list

print('New normalized datetime (first 7 rows):')
pd_df.head(7)[['userId', 'sport', 'date_time', 'normalized_date_time']]

print('\nПостроение необработанного пульс (выборка) по нормированному времени:')

sport_list = pd_df['sport'].unique()
# Динамическое определение длины фигуры зависит от длины спортивного списка.
fig, axs = plt.subplots(len(sport_list), figsize=(15, 6*len(sport_list)))
subplot_adj = fig.subplots_adjust(hspace = 0.6)
plot_setp = plt.setp(axs, yticks=range(0,250,20))

for sport_index, sport in enumerate(sport_list):
```

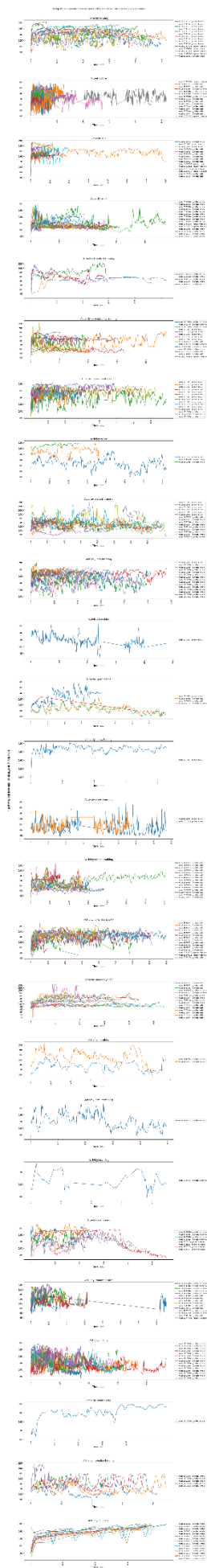
6 сек.выполнено в 10:50

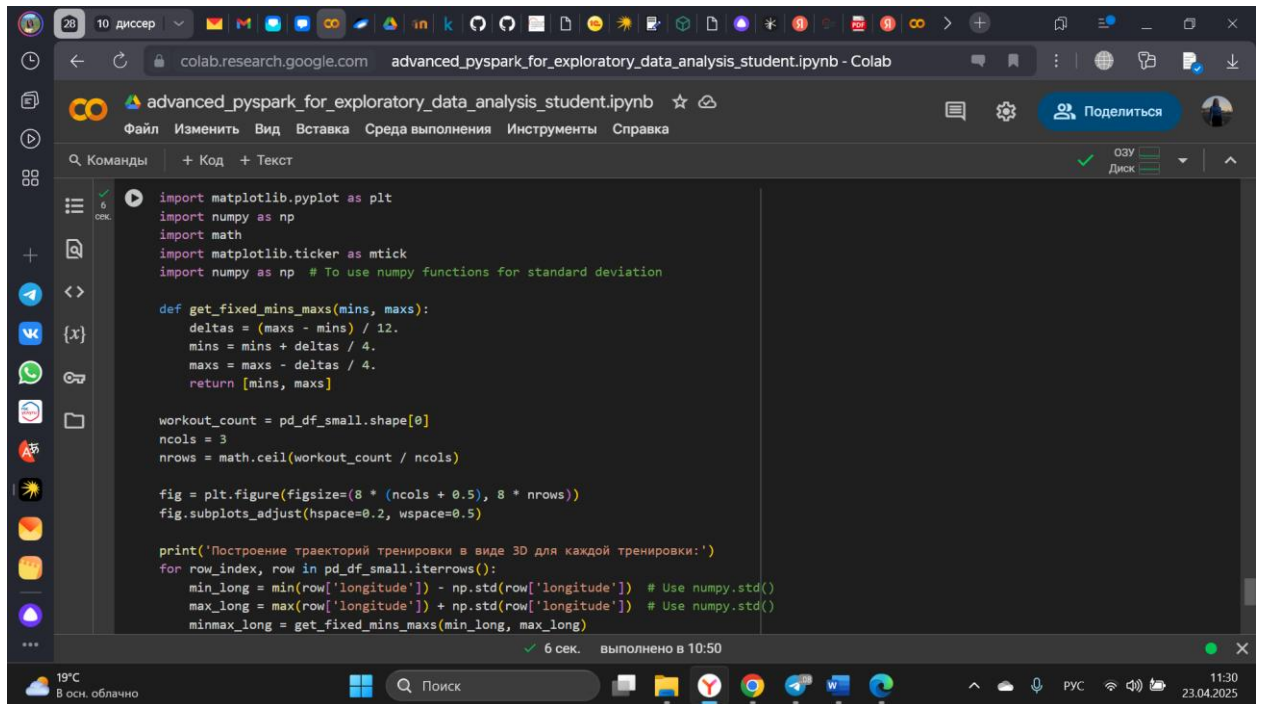
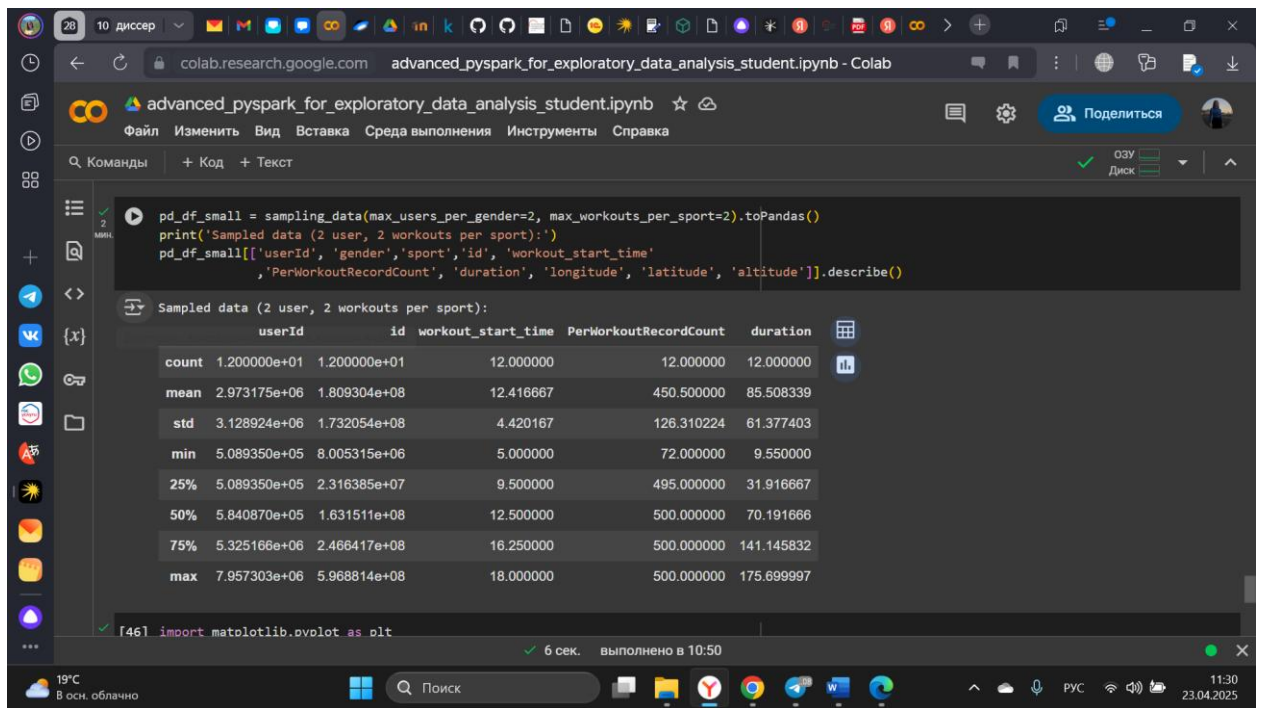
19°CВ осн. облачно

Поиск

Y

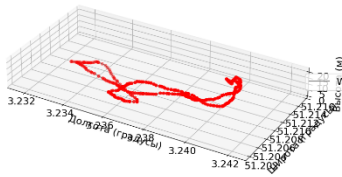
11:3023.04.2025



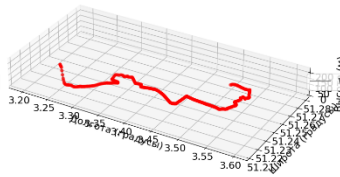


Маршрут тренировки (долгота/широта/высота)

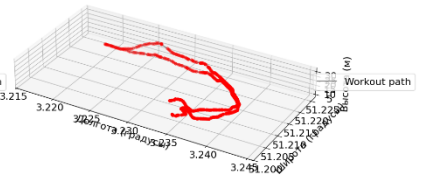
Активность: run - Пол: female
Рекорды: 354 - Длительность: 28 минуты



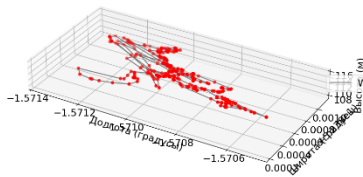
Активность: bike - Пол: female
Рекорды: 500 - Длительность: 66 минуты



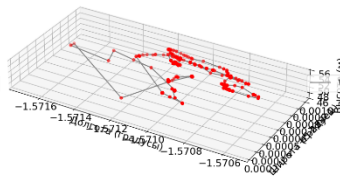
Активность: run - Пол: female
Рекорды: 500 - Длительность: 52 минуты



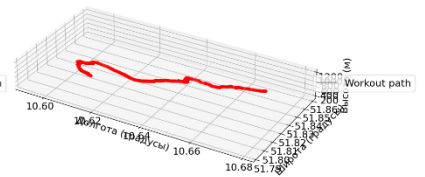
Активность: indoor cycling - Пол: unknown
Рекорды: 500 - Длительность: 73 минуты



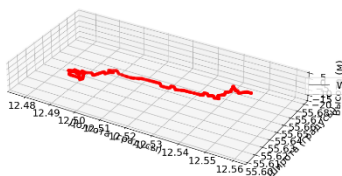
Активность: indoor cycling - Пол: unknown
Рекорды: 500 - Длительность: 91 минуты



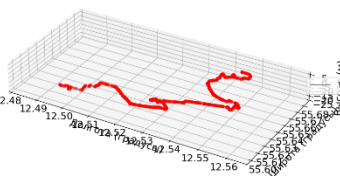
Активность: hiking - Пол: male
Рекорды: 500 - Длительность: 171 минуты



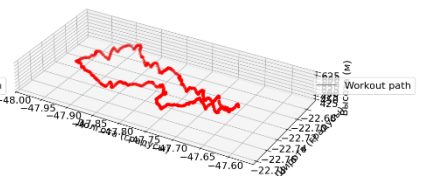
Активность: bike (transport) - Пол: male
Рекорды: 480 - Длительность: 25 минуты



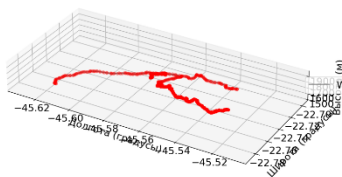
Активность: bike (transport) - Пол: male
Рекорды: 500 - Длительность: 33 минуты



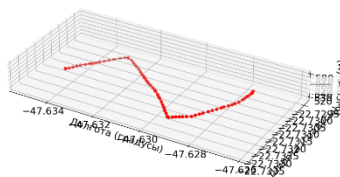
Активность: mountain bike - Пол: male
Рекорды: 500 - Длительность: 175 минуты



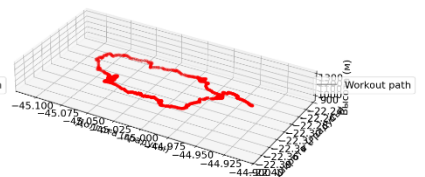
Активность: bike - Пол: male
Рекорды: 500 - Длительность: 132 минуты



Активность: walk - Пол: male
Рекорды: 72 - Длительность: 9 минуты



Активность: mountain bike - Пол: male
Рекорды: 500 - Длительность: 167 минуты



Список теоретических вопросов к отчету:

1. Фреймворк обработки больших данных Apache Spark, его назначение, функции и отличия от Hadoop [MapReduce](#).
2. Понятие устойчивого распределенного набора данных (RDD). Понятие раздела RDD (partition). Способы создания RDD. Трансформации

(transformations) и действия (actions). Кэширование (cache) данных в Spark.

3. RDD и PairRDD: понятие, назначение. Основные трансформации (transformations) и действия (actions) над ними.
4. Реализация концепции [MapReduce](#) в фреймворке Spark. Функции map, flatMap, mapValues, mapPartitions, reduce, reduceByKey.
5. Модели запуска Spark-приложений ([YARN](#), Standalone, Kubernetes). Понятие драйвера (driver) и исполнителей (executors). Понятия задания (job), этапа (stage) и задачи (task). Модель ленивых вычислений (lazy) и ее применение в Spark. Понятие ориентированного ациклического графа (Directed Acyclic Graph, DAG).
6. Операция перемешивания данных (shuffle): причины возникновения, влияние на производительность. Класс Partitioner. Пути повышения производительности.
7. Понятие датафрейма в Spark. Основные операции над датафреймами. Скалярные функции, агрегирующие функции, оконные функции. Оптимизация запросов. Catalyst.
8. Клиентский (client) и кластерный (cluster) режимы работы Apache Spark.

Заключение

В ходе работы были успешно достигнуты все поставленные цели:

1. **Понятие больших данных** было детально изучено, включая их основные характеристики, особенности и методы обработки. Были рассмотрены различные подходы к работе с большими данными, включая распределенные вычисления, параллельную обработку и специализированные инструменты для анализа.
2. **Apache Spark** был детально исследован как современный инструмент для обработки больших данных. Были освоены основные компоненты фреймворка, включая Spark Core, Spark SQL, Spark MLlib и Spark Streaming. Особое внимание было уделено возможностям распределенной обработки данных и масштабируемости системы.
3. **Навыки разведочного анализа** были успешно развиты с использованием PySpark. Были освоены методы предварительной обработки данных, включая фильтрацию, агрегацию и трансформацию. Также были получены практические навыки визуализации данных и выявления основных закономерностей в наборах данных.

В результате работы был сформирован комплексный подход к работе с большими данными, включающий теоретические знания и практические навыки использования современных инструментов обработки данных. Полученные компетенции позволяют эффективно решать задачи анализа и обработки больших массивов данных в реальных проектах.